

- 1.-Instalamos api:
- 2.-Añadimos HasApiTokens al modelo
- 3.-Creamos modelo + mmigración + factory + seeder
- 4.-Editamos la migración, el modelo, el factory, el seeder y el databaseseeder
- 5.- Creamos AuthController y lo editamos
- 6.-Creamos Requests y las editamos
- 7.-Creamos el controlador API y lo editamos
- 8.-Editamos las rutas en routes/api.php

1.-Instalamos api:

phpartisaninstall:api

2.-Añadimos HasApiTokens al modelo

```
class User extends Authenticatable
{
    /** @use HasFactory<\Database\Factories\UserFactory> */
    use HasApiTokens, HasFactory, Notifiable;
```

3.-Creamos modelo + mmigración + factory + seeder

phpartisanmake:modelEventVenue-mfs

4.-Editamos la migración, el modelo, el factory, el seeder y el databaseseeder

```

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('event_venues', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->unsignedInteger('capacity');
            $table->string('city');
            $table->string('country');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('event_venues');
    }
};

```

Ejecutamos php artisan migrate

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class EventVenue extends Model
{
    use HasFactory;

    protected $fillable = ['name', 'capacity', 'city', 'country'];
}

```

```

public function definition(): array
{
    return [
        'name' => 'Venue ' . $this->faker->unique()->word(),
        'capacity' => $this->faker->numberBetween(50, 5000),
        'city' => $this->faker->city(),
        'country' => $this->faker->country(),
    ];
}
}

```

```

class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        User::factory()->create([
            'name' => 'AdminJose',
            'email' => 'adminjose@example.com',
            'password' => '1234',
        ]);

        $this->call([
            EventVenueSeeder::class,
        ]);
    }
}

```

Ejecutamos php artisan db:seed

5.- Creamos AuthController y lo editamos

php artisan make:controller Api/V1/AuthController

```

<?php

namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;

class AuthController extends Controller
{
    public function register(Request $request)
    {
        $data = $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'email', 'max:255', 'unique:users,email'],
            'password' => ['required', 'string', 'min:6'],
        ]);

        $user = User::create([
            'name' => $data['name'],
            'email' => $data['email'],
            'password' => Hash::make($data['password']),
        ]);

        $token = $user->createToken('api-token')->plainTextToken;

        return response()->json([
            'user' => $user,
            'token' => $token,
        ], 201);
    }

    public function login(Request $request)
    {
        $data = $request->validate([
            'email' => ['required', 'email'],
            'password' => ['required', 'string'],
        ]);

        $user = User::where('email', $data['email'])->first();

        if (!$user || !Hash::check($data['password'], $user->password)) {
            throw ValidationException::withMessages([
                'email' => ['Credenciales inválidas.'],
            ]);
        }

        $token = $user->createToken('api-token')->plainTextToken;

        return response()->json(['token' => $token]);
    }

    public function logout(Request $request)
    {
        $request->user()->currentAccessToken()->delete();

        return response()->json(['message' => 'Logout OK']);
    }
}

```

6.-Creamos Requests y las editamos

phpartisanmake:requestStoreEventVenueRequest

phpartisanmake:requestUpdateEventVenueRequest

```

class StoreEventVenueRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true; // El acceso lo controla auth:sanctum
    }

    public function rules(): array
    {
        return [
            'name'      => ['required', 'string', 'max:255'],
            'capacity' => ['required', 'integer', 'min:0'],
            'city'      => ['required', 'string', 'max:255'],
            'country'   => ['required', 'string', 'max:255'],
        ];
    }
}

```

```

class UpdateEventVenueRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'name'      => ['sometimes', 'required', 'string', 'max:255'],
            'capacity' => ['sometimes', 'required', 'integer', 'min:0'],
            'city'      => ['sometimes', 'required', 'string', 'max:255'],
            'country'   => ['sometimes', 'required', 'string', 'max:255'],
        ];
    }
}

```

7.-Creamos el controlador API y lo editamos

phpartisanmake:controllerApi/V1/EventVenueController--api

```

class EventVenueController extends Controller
{
    // Público
    public function index()
    {
        return response()->json(
            EventVenue::query()->latest()->get()
        );
    }

    // Público
    public function show(EventVenue $eventVenue)
    {
        return response()->json($eventVenue);
    }

    // Protegido
    public function store(StoreEventVenueRequest $request)
    {
        $created = EventVenue::create($request->validated());
        return response()->json($created, 201);
    }

    // Protegido
    public function update(UpdateEventVenueRequest $request, EventVenue $eventVenue)
    {
        $eventVenue->update($request->validated());
        return response()->json($eventVenue);
    }

    // Protegido
    public function destroy(EventVenue $eventVenue)
    {
        $eventVenue->delete();
        return response()->json(null, 204);
    }
}

```

8.-Editamos las rutas en routes/api.php

```
<?php
```

```
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\V1\AuthController;
use App\Http\Controllers\Api\V1\EventVenueController;
```

```
Route::prefix('v1')->group(function () {
```

```
    // Auth
```

```
    Route::post('/auth/register', [AuthController::class, 'register']);
```

```
    Route::post('/auth/login', [AuthController::class, 'login']);
```

```
    Route::middleware('auth:sanctum')->post('/auth/logout', [AuthController::class, 'logout']);
```

```
    // EventVenues (público: listar/ver)
```

```
    Route::get('/event-venues', [EventVenueController::class, 'index']);
```

```
    Route::get('/event-venues/{eventVenue}', [EventVenueController::class, 'show']);
```

```
    // EventVenues (protegido: crear/editar/borrar)
```

```
    Route::middleware('auth:sanctum')->group(function () {
```

```
        Route::post('/event-venues', [EventVenueController::class, 'store']);
```

```
        Route::match(['put', 'patch'], '/event-venues/{eventVenue}', [EventVenueController::class, 'update']);
```

```
        Route::delete('/event-venues/{eventVenue}', [EventVenueController::class, 'destroy']);
```

```
    });
```

```
});
```